

Plan de Pruebas

Carolina Vildósola Guzmán A01287373

Para poder comprobar que el programa funciona correctamente hice diferentes pruebas usando los 2 archivos que se nos dieron. Probe cada uno de los algoritmos de ordenamiento para comparar sus tiempos, también hice diferentes búsquedas por rango de fechas. Además probe algunos casos especiales como fechas inválidas, buscar rangos donde no hay resultados, utilizar fechas que estuvieran en los bordes de la búsqueda y comprobar que los timestamps repetidos no se perdieran. Con todas estas pruebas asegure no solo que el programa corra, sino que funcione bien.

Casos de prueba

Caso 1. Rango vacío

Prueba: Se ingresó un rango del 7 de septiembre de 2024 de 00:00:00 a 23:59:59. Este rango es anterior al primer registro del archivo.

Resultado esperado: Que no se encontraran logs dentro del rango y que el programa continuará funcionando correctamente.

Resultado obtenido: No se encontraron logs dentro del rango, como se esperaba.

```
Antes de ordenar, escribe tu prediccion.
Crees que sera rapido o lento?: rapido
Por que?: porque tiene complejidad  $O(n \log n)$ 
Los logs ordenados se guardaron en output607.txt

Ordenamiento terminado.
Algoritmo usado: Merge Sort
Archivo usado: log607-1.txt
Cantidad de logs: 6818
Tiempo: 87598042 nanosegundos
Complejidad teorica:
Merge Sort - Mejor:  $O(n \log n)$  | Peor:  $O(n \log n)$ 
Tu prediccion fue: rapido
Razon: porque tiene complejidad  $O(n \log n)$ 

El resultado coincidio con tu prediccion?
1. Si
2. No
Opcion: 1
El resultado si coincidio con la prediccion inicial.

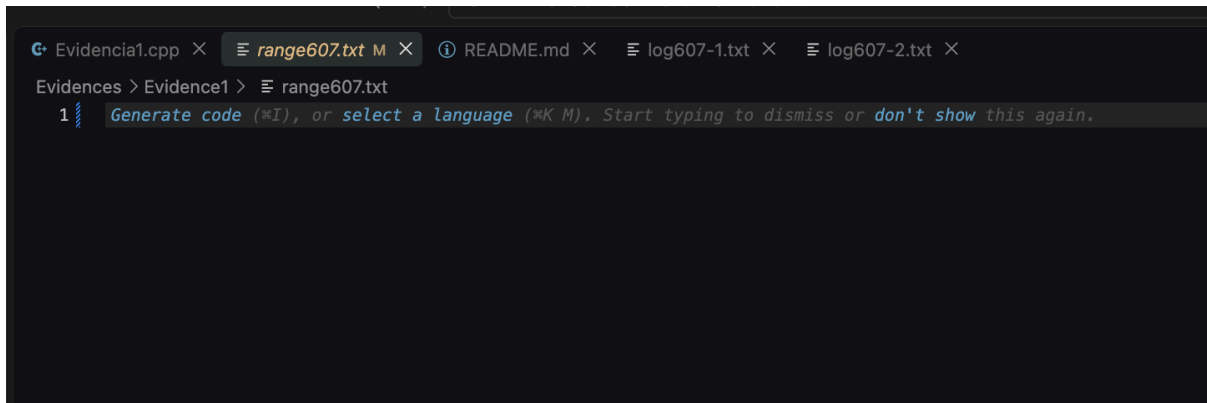
Busqueda por rango de fecha y hora

Escribe la fecha y hora inicial:
Mes (primeras 3 letras, ej. Sep): Sep
Dia: 7
Año: 2024
Hora (HH:MM:SS): 00:00:00

Escribe la fecha y hora final:
Mes (primeras 3 letras, ej. Sep): Sep
Dia: 7
Año: 2024
Hora (HH:MM:SS): 23:59:59

No se encontraron logs dentro de ese rango.
El rango se guardo en range607.txt

Deseas hacer otra corrida?
1. Si
2. No
Opcion: 2
```



Caso 2. Bordes de fechas

Prueba: Se utilizó como fecha inicial y final el mismo timestamp: Sep 8 2024 00:22:43, que corresponde al primer registro del archivo.

Resultado esperado: Que el programa incluyera el registro aunque su fecha y hora fueran exactamente iguales a los límites de la búsqueda.

Resultado obtenido: El programa encontró de manera correcta solo 1 log dentro del rango.

```
ser/s/CarolinaVillalobos/ta/documentos/CMIT-FC1031-007-2013-A01267373/Evidences/Evidence1/ Evidencia1
Crees que sera rapido o lento?: rapido
Por que?: porque tiene complejidad O(n log n)
Los logs ordenados se guardaron en output607.txt

Ordenamiento terminado.
Algoritmo usado: Merge Sort
Archivo usado: log607-1.txt
Cantidad de logs: 6818
Tiempo: 88664584 nanosegundos
Complejidad teorica:
Merge Sort - Mejor: O(n log n) | Peor: O(n log n)
Tu prediccion fue: rapido
Razon: porque tiene complejidad O(n log n)

El resultado coincidio con tu prediccion?
1. Si
2. No
Opcion: 1
El resultado si coincidio con la prediccion inicial.

Busqueda por rango de fecha y hora

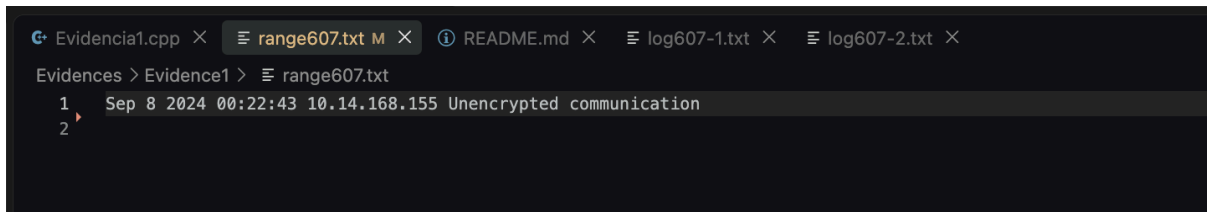
Escribe la fecha y hora inicial:
Mes (primeras 3 letras, ej. Sep): sep
Dia: 8
Año: 2024
Hora (HH:MM:SS): 00:22:43

Escribe la fecha y hora final:
Mes (primeras 3 letras, ej. Sep): sep
Dia: 8
Año: 2024
Hora (HH:MM:SS): 00:22:43

Se encontraron 1 logs dentro del rango.

Logs encontrados:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
El rango se guardo en range607.txt

Deseas hacer otra corrida?
1. Si
2. No
Opcion: 
```



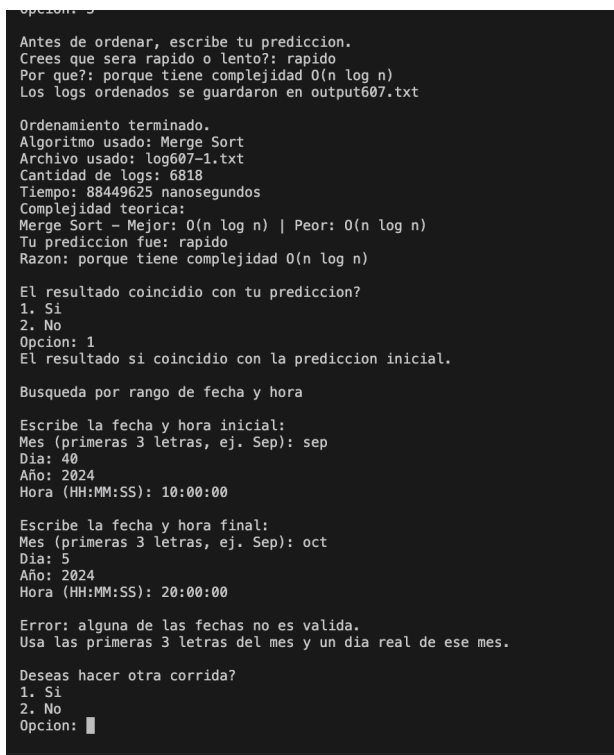
```
Evidencia1.cpp × range607.txt M × README.md × log607-1.txt × log607-2.txt ×
Evidences > Evidence1 > range607.txt
1 Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
2
```

Caso 3. Entrada inválida

Prueba: Se ingresa la fecha Sep 40 2024, la cual no existe.

Resultado esperado: Que el programa detectara que la fecha era inválida y no realizará la búsqueda.

Resultado obtenido: El programa detectó correctamente la fecha inválida y mostró un mensaje de error.



```
opcion: 3
Antes de ordenar, escribe tu prediccion.
Crees que sera rapido o lento?: rapido
Por que?: porque tiene complejidad O(n log n)
Los logs ordenados se guardaron en output607.txt

Ordenamiento terminado.
Algoritmo usado: Merge Sort
Archivo usado: log607-1.txt
Cantidad de logs: 6818
Tiempo: 88449625 nanosegundos
Complejidad teorica:
Merge Sort - Mejor: O(n log n) | Peor: O(n log n)
Tu prediccion fue: rapido
Razon: porque tiene complejidad O(n log n)

El resultado coincidio con tu prediccion?
1. Si
2. No
Opcion: 1
El resultado si coincidio con la prediccion inicial.

Busqueda por rango de fecha y hora

Escribe la fecha y hora inicial:
Mes (primeras 3 letras, ej. Sep): sep
Dia: 40
Año: 2024
Hora (HH:MM:SS): 10:00:00

Escribe la fecha y hora final:
Mes (primeras 3 letras, ej. Sep): oct
Dia: 5
Año: 2024
Hora (HH:MM:SS): 20:00:00

Error: alguna de las fechas no es valida.
Usa las primeras 3 letras del mes y un dia real de ese mes.

Deseas hacer otra corrida?
1. Si
2. No
Opcion: █
```

Caso 4. Timestamps duplicados

Prueba: Se buscó el timestamp Dec 23 2025 06:46:34, el cual aparece dos veces en el archivo log607-2.txt.

Resultado esperado: Que la búsqueda incluyera todos los registros que tuvieran la misma fecha y hora, sin perder ninguno.

Resultado obtenido: El programa encontró correctamente los 2 logs que tienen el mismo timestamp.

```
carolinavildosola@MacBookAir Evidence1 % cd "/Users/carolinavildosola/Do
sers/carolinavildosola/Documents/C.MTY.TC1031.607.2613.A01287373/Evidenc
Por que?: por su complejidad
Los logs ordenados se guardaron en output607.txt

Ordenamiento terminado.
Algoritmo usado: Merge Sort
Archivo usado: log607-2.txt
Cantidad de logs: 6818
Tiempo: 83344042 nanosegundos
Complejidad teorica:
Merge Sort - Mejor:  $O(n \log n)$  | Peor:  $O(n \log n)$ 
Tu prediccion fue: rapido
Razon: por su complejidad

El resultado coincidio con tu prediccion?
1. Si
2. No
Opcion: 1
El resultado si coincidio con la prediccion inicial.

Busqueda por rango de fecha y hora

Escribe la fecha y hora inicial:
Mes (primeras 3 letras, ej. Sep): dec
Dia: 23
Año: 2025
Hora (HH:MM:SS): 06:46:34

Escribe la fecha y hora final:
Mes (primeras 3 letras, ej. Sep): dec
Dia: 23
Año: 2025
Hora (HH:MM:SS): 06:46:34

Se encontraron 2 logs dentro del rango.

Logs encontrados:
Dec 23 2025 06:46:34 10.14.134.39 Social engineering attacks
Dec 23 2025 06:46:34 10.14.112.225 Configuration errors
El rango se guardo en range607.txt

Deseas hacer otra corrida?
1. Si
2. No
Opcion: █
```

Caso 5. Archivo output607.txt

Prueba: Se revisó el archivo output607.txt después de ejecutar el algoritmo de ordenamiento.

Resultado esperado: Que los logs se guardaran ordenados de forma ascendente por fecha y hora.

Resultado obtenido: Los registros se guardaron correctamente en orden cronológico por fecha y hora.

```
Evidencia1.cpp x range607.txt M x output607.txt M x README.md x log607-1.txt
Evidences > Evidence1 > output607.txt
1 Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
2 Sep 8 2024 04:37:37 10.14.248.79 Configuration errors
3 Sep 8 2024 04:44:15 10.14.135.12 Application error
4 Sep 8 2024 05:50:57 10.14.22.235 Changes in File Integrity
5 Sep 8 2024 10:42:42 10.14.40.195 Data corruption
6 Sep 8 2024 10:47:52 10.14.24.223 Data corruption
7 Sep 8 2024 15:23:16 10.14.206.243 Weak or easily guessable passwords
8 Sep 8 2024 15:25:34 10.14.220.55 Broken authentication and session management
9 Sep 9 2024 01:51:20 10.14.27.106 Application error
10 Sep 9 2024 04:53:57 10.14.96.200 Suspicious IP Addresses
11 Sep 9 2024 07:53:44 10.14.230.164 Data Exfiltration Attempts
12 Sep 9 2024 15:18:08 10.14.182.26 Suspicious IP Addresses
13 Sep 9 2024 18:59:55 10.14.132.80 Failed Login Attempts
14 Sep 9 2024 19:17:57 10.14.186.180 Cross-Site Scripting
15 Sep 10 2024 09:16:12 10.14.245.197 Suspicious IP Addresses
16 Sep 10 2024 13:50:52 10.14.99.247 Anomalous User Activity
17 Sep 10 2024 18:14:07 10.14.153.89 Anomalous User Activity
18 Sep 10 2024 21:18:36 10.14.183.213 Data Exfiltration Attempts
19 Sep 11 2024 06:44:21 10.14.80.172 Suspicious IP Addresses
20 Sep 11 2024 08:48:35 10.14.45.122 Anomalous User Activity
21 Sep 11 2024 13:23:41 10.14.241.4 Changes in File Integrity
22 Sep 11 2024 17:10:13 10.14.138.230 Encryption Failures
23 Sep 12 2024 00:25:50 10.14.18.96 Human error
24 Sep 12 2024 03:30:41 10.14.52.18 SQL injection
25 Sep 12 2024 03:31:18 10.14.13.16 Changes in File Integrity
26 Sep 12 2024 03:58:23 10.14.125.11 Encryption Failures
27 Sep 12 2024 04:03:41 10.14.79.25 Application error
28 Sep 12 2024 05:48:17 10.14.37.249 Malware Detection Alerts
29 Sep 12 2024 06:35:19 10.14.211.220 Unauthorized Access Attempts
30 Sep 12 2024 11:59:08 10.14.226.221 Application error
31 Sep 12 2024 12:54:23 10.14.252.43 Security breaches
32 Sep 12 2024 14:37:01 10.14.55.63 Security breaches
33 Sep 12 2024 16:27:42 10.14.54.112 Load balancing issues
34 Sep 12 2024 19:11:50 10.14.165.115 Unauthorized Access Attempts
35 Sep 12 2024 21:23:06 10.14.17.242 Cross-Site Scripting
36 Sep 12 2024 21:24:52 10.14.51.28 Configuration errors
37 Sep 13 2024 03:23:54 10.14.60.174 Cross-Site Scripting
```

Caso 6. Archivo range607.txt

Prueba: Se revisó el archivo range607.txt después de realizar la búsqueda de un timestamp duplicado.

Resultado esperado: Que el archivo guardara todos los logs encontrados dentro del rango.

Resultado obtenido: El archivo guardó correctamente los 2 logs encontrados en la búsqueda.

```
Evidencia1.cpp x range607.txt M x README.md x log607-1.txt
Evidences > Evidence1 > range607.txt
1 Dec 23 2025 06:46:34 10.14.134.39 Social engineering attacks
2 Dec 23 2025 06:46:34 10.14.112.225 Configuration errors
3
```

Resultados de tiempos

Tabla de las 14 corridas

Algoritmo	log607-1.txt	log607-2.txt
Swap Sort	<pre>Elige el archivo: 1. log607-1.txt 2. log607-2.txt Opcion: 1 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 1 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: lento Por que?: porque tiene complejidad O(n^2) Los logs ordenados se guardaron en output607.txt Ordenamiento terminado. Algoritmo usado: Swap Sort Archivo usado: log607-1.txt Cantidad de logs: 6818 Tiempo: 16198899667 nanosegundos Complejidad teorica: Swap Sort - Mejor: O(n^2) Peor: O(n^2) Tu prediccion fue: lento Razon: porque tiene complejidad O(n^2) El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial.</pre>	<pre>Elige el archivo: 1. log607-1.txt 2. log607-2.txt Opcion: 2 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 1 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: lento Por que?: porque sigue haciendo O(n^2) comparaciones Los logs ordenados se guardaron en output607.txt Ordenamiento terminado. Algoritmo usado: Swap Sort Archivo usado: log607-2.txt Cantidad de logs: 6818 Tiempo: 632011458 nanosegundos Complejidad teorica: Swap Sort - Mejor: O(n^2) Peor: O(n^2) Tu prediccion fue: lento Razon: - porque sigue haciendo O(n^2) comparaciones El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial.</pre>
Bubble Sort	<pre>Elige el archivo: 1. log607-1.txt 2. log607-2.txt Opcion: 1 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 2 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: lento Por que?: porque sigue haciendo O(n^2) comparaciones Los logs ordenados se guardaron en output607.txt Ordenamiento terminado. Algoritmo usado: Bubble Sort Archivo usado: log607-1.txt Cantidad de logs: 6818 Tiempo: 19308750508 nanosegundos Complejidad teorica: Bubble Sort - Mejor: O(n^2) Peor: O(n^2) Tu prediccion fue: lento Razon: porque tiene complejidad O(n^2) El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial.</pre>	<pre>Elige el archivo: 1. log607-1.txt 2. log607-2.txt Opcion: 2 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 2 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: lento Por que?: porque sigue haciendo O(n^2) comparaciones Los logs ordenados se guardaron en output607.txt Ordenamiento terminado. Algoritmo usado: Bubble Sort Archivo usado: log607-2.txt Cantidad de logs: 6818 Tiempo: 9238991858 nanosegundos Complejidad teorica: Bubble Sort - Mejor: O(n^2) Peor: O(n^2) Tu prediccion fue: lento Razon: porque sigue haciendo O(n^2) comparaciones El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial.</pre>
Selection Sort	<pre>Elige el archivo: 1. log607-1.txt 2. log607-2.txt Opcion: 1 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 3 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: lento Por que?: porque tiene complejidad O(n^2) Los logs ordenados se guardaron en output607.txt Ordenamiento terminado. Algoritmo usado: Selection Sort Archivo usado: log607-1.txt Cantidad de logs: 6818 Tiempo: 646295968 nanosegundos Complejidad teorica: Selection Sort - Mejor: O(n^2) Peor: O(n^2) Tu prediccion fue: lento Razon: porque tiene complejidad O(n^2) El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial.</pre>	<pre>Elige el archivo: 1. log607-1.txt 2. log607-2.txt Opcion: 2 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 3 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: lento Por que?: porque mantiene complejidad O(n^2) aunque este casi ordenado Los logs ordenados se guardaron en output607.txt Ordenamiento terminado. Algoritmo usado: Selection Sort Archivo usado: log607-2.txt Cantidad de logs: 6818 Tiempo: 646295968 nanosegundos Complejidad teorica: Selection Sort - Mejor: O(n^2) Peor: O(n^2) Tu prediccion fue: lento Razon: porque mantiene complejidad O(n^2) aunque este casi ordenado El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial.</pre>
Insertion Sort	<pre>Elige el archivo: 1. log607-1.txt 2. log607-2.txt Opcion: "R" Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 4 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: lento Por que?: porque el archivo esta desordenado y puede acercarse a O(n^2) Los logs ordenados se guardaron en output607.txt Ordenamiento terminado. Algoritmo usado: Insertion Sort Archivo usado: log607-1.txt Cantidad de logs: 6818 Tiempo: 346937288 nanosegundos Complejidad teorica: Insertion Sort - Mejor: O(n) Peor: O(n^2) Tu prediccion fue: lento Razon: porque el archivo esta desordenado y puede acercarse a O(n^2) El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial.</pre>	<pre>Elige el archivo: 1. log607-1.txt 2. log607-2.txt Opcion: 2 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 4 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: rapido Por que?: porque el archivo esta casi ordenado y se acerca a su mejor caso O(n) Los logs ordenados se guardaron en output607.txt Ordenamiento terminado. Algoritmo usado: Insertion Sort Archivo usado: log607-2.txt Cantidad de logs: 6818 Tiempo: 287335 nanosegundos Complejidad teorica: Insertion Sort - Mejor: O(n) Peor: O(n^2) Tu prediccion fue: rapido Razon: porque el archivo esta casi ordenado y se acerca a su mejor caso O(n) El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial.</pre>

Merge Sort	<pre> Elige el archivo: 1. log67-1.txt 2. log67-2.txt Opcion: 1 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 5 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: rapido Por que?: porque tiene complejidad O(n log n) Los logs ordenados se guardaron en output67.txt Ordenamiento terminado. Algoritmo usado: Merge Sort Archivo usado: log67-1.txt Cantidad de logs: 6818 Tiempo: 0.7860751 nanosegundos Complejidad teorica: Merge Sort = Mejor: O(n log n) Peor: O(n log n) Tu prediccion fue: rapido Razon: porque tiene complejidad O(n log n) El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial. </pre>	<pre> Elige el archivo: 1. log67-1.txt 2. log67-2.txt Opcion: 2 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 5 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: rapido Por que?: porque mantiene complejidad O(n log n) Los logs ordenados se guardaron en output67.txt Ordenamiento terminado. Algoritmo usado: Merge Sort Archivo usado: log67-2.txt Cantidad de logs: 6818 Tiempo: 0.5311669 nanosegundos Complejidad teorica: Merge Sort = Mejor: O(n log n) Peor: O(n log n) Tu prediccion fue: rapido Razon: porque mantiene complejidad O(n log n) El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial. </pre>
Quick Sort	<pre> Elige el archivo: 1. log67-1.txt 2. log67-2.txt Opcion: 1 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 6 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: rapido Por que?: porque en promedio tiene complejidad O(n log n) Los logs ordenados se guardaron en output67.txt Ordenamiento terminado. Algoritmo usado: Quick Sort Archivo usado: log67-1.txt Cantidad de logs: 6818 Tiempo: 0.100675 nanosegundos Complejidad teorica: Quick Sort = Mejor: O(n log n) Peor: O(n^2) Tu prediccion fue: rapido Razon: porque en promedio tiene complejidad O(n log n) El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial. </pre>	<pre> Elige el archivo: 1. log67-1.txt 2. log67-2.txt Opcion: 2 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 6 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: lento Por que?: porque usa el ultimo elemento como pivote y el archivo esta casi ordenado Los logs ordenados se guardaron en output67.txt Ordenamiento terminado. Algoritmo usado: Quick Sort Archivo usado: log67-2.txt Cantidad de logs: 6818 Tiempo: 0.0033719 nanosegundos Complejidad teorica: Quick Sort = Mejor: O(n log n) Peor: O(n^2) Tu prediccion fue: lento Razon: porque usa el ultimo elemento como pivote y el archivo esta casi ordenado El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial. </pre>
Shell Sort	<pre> Elige el archivo: 1. log67-1.txt 2. log67-2.txt Opcion: 1 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 7 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: rapido Por que?: porque reduce las comparaciones usando saltos Los logs ordenados se guardaron en output67.txt Ordenamiento terminado. Algoritmo usado: Shell Sort Archivo usado: log67-1.txt Cantidad de logs: 6818 Tiempo: 0.126725 nanosegundos Complejidad teorica: Shell Sort = Depende de la secuencia de gaps utilizada Tu prediccion fue: rapido Razon: porque reduce las comparaciones usando saltos El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial. </pre>	<pre> Elige el archivo: 1. log67-1.txt 2. log67-2.txt Opcion: 2 Archivo abierto correctamente. Se leyeron 6818 logs. Elige el algoritmo de ordenamiento: 1. Swap Sort 2. Bubble Sort 3. Selection Sort 4. Insertion Sort 5. Merge Sort 6. Quick Sort 7. Shell Sort Opcion: 7 Antes de ordenar, escribe tu prediccion. Crees que sera rapido o lento?: rapido Por que?: porque usa saltos y el archivo esta casi ordenado Los logs ordenados se guardaron en output67.txt Ordenamiento terminado. Algoritmo usado: Shell Sort Archivo usado: log67-2.txt Cantidad de logs: 6818 Tiempo: 0.002625 nanosegundos Complejidad teorica: Shell Sort = Depende de la secuencia de gaps utilizada Tu prediccion fue: rapido Razon: porque usa saltos y el archivo esta casi ordenado El resultado coincidio con tu prediccion? 1. Si 2. No Opcion: 1 El resultado si coincidio con la prediccion inicial. </pre>

Conclusión

Después de hacer todas las pruebas pude comprobar que el programa funciona correctamente tanto para ordenar los logs como para hacer búsquedas por rangos de fechas y horas. También pude ver como el rendimiento cambiaba en base al algoritmo y de como están acomodados los datos! pude entender mucho mejor los tipos de algoritmos.

En el archivo 1, el desordenado los algoritmos más lentos fueron Swap Bubble y Selection, mientras que Merge Quick y Shell tuvieron menores tiempos. Algo que se me hizo interesante fue Insertion Sort ya que el archivo desordenado se tardó aproximadamente 3.42 seg pero al archivo que estaba casi ordenado solo tardo 0.025 segundos. Con eso pude ver mejor cómo afecta el estado inicial en el que están ordenados los datos.

Pasó algo similar con Quick Sort pero al revés, en el archivo desordenado fue muy rápido con 0.082 seg y en el casi ordenado fueron 1.83 seg. Eso es por el uso del último elementos como pivote por lo que cuando se trabaja con datos casi ordenados es su peor caso.

Para concluir, las pruebas de rango vacío, fechas invalida, limited y timestamps repetidos fueron funcionales y con esto pude comprobar que el programa no solo ordena datos sino que también funciona bajo diferentes casos que podrían pasar al momento en que se hace una búsqueda.